

金仓数据库适配：AI 辅助研发经验总结

项目：ThingsBoard 4.1 适配人大金仓 KingbaseES V8 (V008R006C009B0014) 投入：约 **2 周 / 10 人日** (纯人工评估 32 人日) | 工具：Claude Code | 面向：研发中心内部

一、结果

指标	纯人工（评估）	本次
工作量	32 人日	10 人日，效率约 3 倍
业务代码改动	需改查询 + 方言	0 行 （只加 1 个 SQL 函数 + 安装期自动处理）
对原 PostgreSQL 的影响	难以证明	实测为零
验证覆盖	抽测关键场景	950 个用例 × 4 个环境 × 多轮

为什么能这么快？不是 AI 写代码快，而是每一步我都换了一种做法。下面逐步对比。

分工原则：AI 负责执行（装环境、跑测试、写代码、写文档），人负责定边界、做取舍、真实运行验证和问题定性。

二、我做了什么：和常规做法逐步对比

#	环节	别人一般会怎么做	我是怎么做的	效果
1	装环境	自己查手册装金仓、申请授权、逐个初始化三种模式，遇到问题再去搜	让 AI 去装 ：把测试机凭据交给 Claude Code，它自己 SSH 登录安装、放授权、切模式、排障。给的库连不通时，它自己报告并换了另一台继续	原来 6 天的环境准备和熟悉工作大幅压缩，人不用中转每一条命令
2	定版本	手头有哪个版本就装哪个	对齐到客户的精确小版本 。早期在 C007 上验证"零改动能跑通"，换成客户的 C009 后 448 个用例失败，而且 C007 只有两种模式	结论建立在客户真实版本上，避免整体返工
3	提需求	直接说"帮我适配金仓"	先定边界 ：第一句就要求"可以开发，但必须保证不影响原有功能"	所有方案都往最小改动走，最终 0 行业务代码改动
4	验证和判断	AI 改完代码后自己手测几个场景，看报错凭经验判断	让 AI 自己验证、自己判断 ：它跑全量测试、比对结果，用数据给结论，人只看结论	判断依据从"感觉没问题"变成"950 个用例逐项一致"
5	找问题	看到测试失败就去修	先建基线 ：用未修改的原始代码在 PG 和金仓上各跑一轮，只看"基线没有、现在有"的失败	排除 10 个和金仓无关的老问题，避免为假问题做适配

#	环节	别人一般会怎么做	我是怎么做的	效果
6	定根因	一条条报错去查	让 AI 从 2740 条报错里 归纳共同根因	2740 条报错归结到 1 个原因（concat 函数被遮蔽），一个 3 行 SQL 函数解决
7	选驱动和方言	用金仓官方驱动，看上去理所当然	让 AI 实测对比 ：金仓驱动需要配套改方言，工作量大；PG 官方驱动加原 PG 方言就能支持 pg 和 oracle 两种模式	避开了方言改造这部分工作量
8	定范围	AI 做出来、能跑通就交付	质疑改动量，主动砍范围 ：三模式方案 AI 约 10 小时做完，改了 165 处查询加方言层。我评估风险后叫停，一期只做 pg 和 oracle	改动量从 165 处查询降到 0 行，上线风险大幅降低
9	理解方案	AI 说什么就信什么	追问到自己能讲清楚 ：方言选型依据、连接串的含义都是追问后才补全的；review 时还发现 AI 按产品名称判断金仓的逻辑实际没生效	能对外讲清楚每个决策，也拦下了隐蔽 bug
10	回归测试	改完测一遍，或只测改动的部分	每次改动都跑矩阵 ：PG、金仓 pg、金仓 oracle 全量回归；单轮 17 分钟，挂后台跑，跑完看结论	"对 PG 零影响"有数据证明
11	交付代码	直接拿探索分支提交	从 master 重开干净分支 ，只合有用的代码。本次重开两次	最终提交只有 7 个文件，评审和合并都很轻
12	交付质量	写文档让实施人员手工执行补丁	安装期自动处理 ：自动探测模式、检查权限、执行补丁、复验结果，不通过就中断并提示怎么修	实施人员不需要懂金仓，问题不会留到现场
13	真实运行验证	测试通过就认为完成	人工亲自编译、启动、运行 ，走一遍真实业务；问题反馈给 AI 前，先 判断是用法不对还是真有缺陷	测试覆盖不到的问题提前暴露；用法问题改文档，真缺陷才改代码
14	跨会话	每开一个新会话都重讲一遍背景	结论沉淀 ：用 CLAUDE.md 记项目背景，用记忆文件记实测结论和版本号	跨十几个会话不丢上下文，也没有把版本差异误判成代码退化

三、换一个人来做行不行

行，但要看怎么做。

做法	预计结果
纯人工	约 32 人日（安装环境 2 天、申请授权 1 天、初始化三种模式 2 天、熟悉金仓 1 天、摸底 + 开发 20 天、自测修复 6 天）
用了 AI，但只是让 AI "去适配"	很可能走到本次一度出现的三模式大改方案：165 处查询加方言层。代码写得快，但评审、回归、上线风险的成本都转嫁到后面，对原有功能有没有影响也说不清
用 AI，并按本次方法做	可以接近本次的效率。组里都熟悉 PG，这次我自己之前也不熟悉金仓，说明方法不依赖个人对金仓的经验

换人做最容易卡住的地方：

- 1. 环境装好后不知道从哪下手，开始一个个查报错。
- 2. 没有先定"不影响原有功能"的边界，AI 的改动面越铺越大。
- 3. 不知道要先跑基线，把老问题当成金仓问题来修。
- 4. AI 跑通了就以为完成了，没有质疑改动量和风险。
- 5. 测试全绿就放心，没有追问"它到底走没走那条路径"。

换人要具备的条件：熟悉 PostgreSQL；会用 Claude Code 这类能直接执行命令的 AI 工具；有一台可以交给 AI 操作的测试机；关键决策由人把关；**能自己编译、启动系统做真实验证，并判断问题是用法不对还是真有缺陷。**

四、提炼出的经验

- 1. **能交给 AI 动手的，都交给它动手。** 装环境、跑测试、看日志、排障，让 AI 直接上机执行，人不做中转。
- 2. **能让 AI 验证的，都让它用数据说话。** 不让 AI 凭知识回答，结论都要有实测结果支撑。
- 3. **先定边界，再让 AI 动手。** "不影响原有功能"这一条决定了整个方案的方向。
- 4. **先建基线，只看差异。** 这是本次最省时间的一招，适用于所有迁移和适配类工作。
- 5. **AI 负责把方案做出来，人负责判断做不做。** AI 不会主动建议砍需求，范围和风险必须由人来定。
- 6. **看不懂就追问，测试全绿也要多问一句。** 最危险的问题都是不报错的。
- 7. **探索和交付分开。** 探索可以随便试，交付必须从主干重开干净分支。
- 8. **最后一公里必须人工验证。** 测试全绿不等于能用。人要亲自编译、启动、运行系统走一遍业务；发现问题反馈给 AI 之前，先判断是使用不当还是真有缺陷：用法问题改文档或配置，真缺陷才改代码。

五、下次直接复用：适配清单

适用于其他国产数据库（达梦、海量等）和中间件的适配：

步骤	做什么	提示词示例
1	确认客户的 精确版本 （含小版本）和兼容模式	—
2	准备测试机，让 AI 安装同版本并初始化所有模式	"机器凭据给你，帮我装好 <版本>，初始化 <模式>，你直接验证给我结果"
3	定边界	"可以开发，但必须保证不影响原有功能"
4	跑基线	"先用没改过的原始代码在 <原数据库> 和 <新数据库各模式> 跑全量测试，作为基线。后台跑，跑完告诉我"
5	归因	"对比基线，只列出新增的失败，并归纳根因"
6	定方案	"给我两到三个方案的对比，说明改动量和风险，你更推荐哪个？如果不考虑 <某个范围>，是不是就不用改代码了？"
7	开发 + 矩阵回归	"每次改完都在所有环境跑一遍全量测试，只给我和基线的差异"
8	追问 + review	"这个地方我没看懂"；"review 本次改动，对原有路径有什么影响"
9	干净交付	"基于最新的 master 新建分支，只合有用的代码，验证脚本不要"

步骤	做什么	提示词示例
10	人工编译、启动、运行验证	人亲自走一遍业务。发现问题先定性：是使用不当，还是真有缺陷？再反馈给 AI："启动后 <现象>，我的配置是 <配置>，帮我判断是配置问题还是代码问题"
11	真实环境实跑 + 文档	"基于实际代码生成部署文档"
12	沉淀	"把这次的实测结论和版本号记下来"

人必须亲自把住的五件事：

- 1. 改动范围和风险。
- 2. 测试全绿时，逻辑是否真的生效。
- 3. 换版本后，旧结论是否还成立。
- 4. 涉及业务语义的变化（如大小写不敏感），要交业务确认。
- 5. 真实编译、启动、运行验证，并对发现的问题定性：是使用不当还是真有缺陷，决定改文档还是改代码。

附：关键数据

最终验证矩阵（950 个用例，PostgreSQL 15.4 为基线）：

环境	代码	Failures	Errors	与基线对比
PostgreSQL 15.4	原始代码	4	6	基线
金仓 pg 模式	未改动	4	6	逐项一致
金仓 oracle 模式	未改动、无补丁	128	320	+441
金仓 oracle 模式	未改动、有补丁	4	6	逐项一致

产出物：交付分支 `feature/kingbase-V008R006C009B0014-pg-oracle`；部署说明 `docs/kingbase-pg-oracle.md`；一键部署 `docker/kingbase/docker-compose.yml`。